UNIVERSIDAD DE CARTAGENA
Programa de Ingeniería de Software

Sistema de nómina orientado a objetos

ACTIVIDAD UNIDAD TRES
INGENIERÍA DE SOFTWARE

Jesus Davod Campo
Autor

Cartagena de Indias
1 de noviembre de 2026

Repositorio del proyecto
https://github.com/tab-Jesus/IngDeSoftware

1. Propósito y requisitos

Objetivo general. Desarrollar un sistema de nómina orientado a objetos que aplique reglas diferenciadas por modalidad de empleado y permita verificar los resultados mediante pruebas unitarias. **Objetivos específicos:** separar responsabilidades con SOLID, validar los datos de entrada y generar comprobantes reproducibles con desglose de conceptos.

El sistema liquida ingresos, bonos, deducciones y beneficios para cuatro modalidades de empleados. Su diseño separa reglas de negocio, cálculo de descuentos y presentación, de modo que el comportamiento puede extenderse y verificarse sin depender de una interfaz gráfica.

Requisito de la guía	Implementación y aceptación
Asalariado	Salario mensual fijo. Bono del 10% únicamente con antigüedad mayor a cinco años.
Por horas	Hasta 40 horas a tarifa normal; exceso a 1.5 veces. Sin bono monetario.
Por comisión	Base más ventas por porcentaje. Bono del 3% de todas las ventas si superan $20.000.000.
Temporal	Salario fijo; fechas inicial y final válidas. Sin bonos ni alimentación adicional.
Descuentos y ARL	Política inyectada con tasas configurables y desglose por concepto.
Beneficios	Alimentación de $1.000.000 para permanentes. Fondo del 2% para empleados por horas elegibles que acepten.
Validaciones	Rechazo de valores negativos o no finitos, contratos invertidos, tasas inválidas y netos negativos.
Ingeniería	POO, SOLID, código limpio, refactorización, pruebas y metodología documentados.

Entradas: identificación, nombre, antigüedad y parámetros de la modalidad. **Salidas:** pago ordinario, bono, bruto, seguro, pensión, ahorro, neto, alimentación, ARL y costo modelado para la empresa.

2. Supuestos y fórmulas

La guía contiene detalles no definidos. Para que los cálculos sean reproducibles se adoptan las siguientes decisiones académicas, ajustables si el docente especifica otra interpretación:

Concepto	Decisión explícita
Seguro y pensión	Se interpreta 4% para seguro y 4% para pensión. Si el 4% fuera combinado, las tasas se pueden modificar en DeductionPolicy.
ARL	Se registra como costo empresarial separado. La tasa de 0.522% es ilustrativa y configurable, pues la guía no fija tarifa ni pagador.
Alimentación	Beneficio empresarial separado del neto y del bruto usado como base de deducciones en este modelo.
Fondo de ahorro	Retención del 2% del bruto, depositada al fondo; requiere más de un año y aceptación. No se presume aporte adicional de la empresa.
Períodos	Las horas se comparan con 40 en el período ingresado. No se inventa conversión semanal a mensual. El temporal exige fecha dentro del contrato, sin prorrateo.
Precisión	Decimal y redondeo HALF_UP a dos decimales por concepto; bruto = ordinario redondeado + bono redondeado.

Fórmulas comunes

Seguro = bruto × tasa de seguro.
Pensión = bruto × tasa de pensión.
Ahorro = bruto × 2%, solo si corresponde.
Neto = bruto - seguro - pensión - ahorro.
Costo modelado de empresa = bruto + alimentación + ARL.

Estos supuestos responden a un ejercicio de programación. No constituyen una liquidación laboral certificada ni un cálculo completo de obligaciones de una empresa.

3. Diseño y programación orientada a objetos

Abstracción: Employee es una clase abstracta que define earnings y el contrato común. **Herencia:** Salaried, Hourly, Commission y Temporary especializan sus reglas. **Polimorfismo:** PayrollService solicita las mismas operaciones a cualquier empleado. **Encapsulación:** entidades inmutables y validación al construirlas mantienen estados válidos.

Capa / clase	Responsabilidad
Employee	Contrato: earnings(), validate_period(), food y saving_rate.
Salaried / Hourly / Commission / Temporary	Reglas específicas de ingresos, beneficios y elegibilidad.
DeductionPolicy	Tasas, deducciones y aporte ARL de la empresa.
PayrollService	Valida período, solicita ingresos, aplica política y verifica neto.
Payslip	Resultado inmutable del cálculo.
__main__.py	Ejemplos ejecutables, presentación por consola y exportación CSV.

Relaciones del modelo

Employee <-- Salaried, Hourly, Commission, Temporary
PayrollService --> Employee
PayrollService --> DeductionPolicy
PayrollService --> Payslip
Consola / CSV --> PayrollService

Flujo: crear empleado válido → validar fecha → calcular ordinario y bono → obtener bruto → aplicar deducciones y ahorro → rechazar neto negativo → devolver comprobante → presentar o exportar.

Para incorporar otra modalidad se implementa Employee y se reutiliza el servicio. La interfaz no contiene reglas de cálculo; las pruebas pueden ejecutarse sin exportar archivos.

4. SOLID, limpieza y refactorización

Principio	Aplicación concreta
S · Responsabilidad única	Entidades para ingresos, política para tasas, servicio para orquestación y consola para salida.
O · Abierto/cerrado	No hay una cadena de condicionales por tipo en el servicio. Cada variante implementa su cálculo y validación de período.
L · Sustitución	Cada variante satisface el mismo contrato de ingresos y beneficios. Se permiten errores de elegibilidad documentados para períodos inválidos.
I · Segregación	El contrato del dominio es pequeño y no obliga a los empleados a imprimir, exportar CSV ni almacenar datos.
D · Inversión de dependencias	El servicio recibe la política en el constructor y opera con el contrato Employee. Otra política compatible puede sustituirse sin cambiar el cálculo de empleados.

Código limpio. Nombres explícitos, funciones cortas, comentarios que explican decisiones, constantes decimales y validaciones cercanas a los datos. Las dataclasses reducen código repetido y expresan claramente los atributos del modelo.

Refactorización aplicada. La comprobación del contrato temporal se trasladó desde una consulta del tipo concreto en el servicio hacia validate_period en las entidades. Con ello se eliminó la dependencia del servicio respecto a Temporary. También se concentraron las tasas en DeductionPolicy y el redondeo en money().

Verificación de la refactorización. Se conservaron oráculos numéricos independientes para comprobar que la reorganización de responsabilidades no modificara los resultados de las reglas de negocio. Las pruebas diferencian el neto del trabajador del costo empresarial e incluyen los límites estrictos de antigüedad, horas y ventas.

5. Resultados y pruebas unitarias

Se ejecutaron 25 pruebas con unittest. Los casos verifican resultados calculados manualmente, límites estrictos, entradas inválidas y configuración alternativa. El registro reproducible está en evidencia/pruebas.txt.

Ejemplo ficticio	Bruto	Neto	Costo empresa
Asalariado: $3 millones, 6 años	$3.300.000	$3.036.000	$4.317.226
Horas: 45 × $20.000, ahorro	$950.000	$855.000	$954.959
Comisión: base $2 millones, ventas $25 millones, 5%	$4.000.000	$3.680.000	$5.020.880
Temporal: salario $1.800.000	$1.800.000	$1.656.000	$1.809.396

Verificación manual del empleado por horas: 40 × $20.000 + 5 × $30.000 = $950.000. Seguro: $38.000. Pensión: $38.000. Ahorro: $19.000. Neto: $855.000. ARL empresarial: $4.959. Sin alimentación ni bono.

Familia de pruebas	Comportamiento esperado
Fronteras	5 años: sin bono; más de 5: con bono. 40 horas: sin extras. $20 millones: sin bono adicional. Un año: sin ahorro.
Validaciones	Horas, ventas y salarios negativos o no finitos se rechazan; nombres vacíos, tasas fuera de rango y antigüedad negativa también.
Contrato y consistencia	Fechas inclusivas válidas; período externo e inversión de fechas rechazados. Neto negativo rechazado; cero permitido.
Dinero y política	Redondeo 1.005 a 1.01; alimentación separada; tasas alternativas 2% + 2% producen neto 96 sobre bruto 100.

Resultado observado: 25 pruebas aprobadas, sin fallos. No se afirma cobertura porcentual ni pruebas de integración bancaria. Cada cálculo individual usa un número fijo de operaciones: tiempo O(1); procesar n empleados requiere O(n) y la exportación actual conserva O(n) resultados en memoria.

6. Metodología y trazabilidad

Se aplicó desarrollo incremental inspirado en Scrum, adaptado a una elaboración individual. No se atribuyen reuniones, roles grupales ni participación CIPA que no se hayan comprobado.

Incremento	Entregable y criterio de aceptación
1 · Análisis	Backlog: cuatro modalidades, deducciones, beneficios y validaciones. Aceptación: cada regla tiene una decisión explícita.
2 · Construcción	Modelo POO, política, servicio y CLI. Aceptación: los cuatro ejemplos devuelven desglose coherente y CSV.
3 · Verificación	Pruebas de cálculo, fronteras y errores. Aceptación: 25 pruebas pasan con oráculos numéricos independientes.
4 · Entrega	Informe, README, guion y paquete. Aceptación: instrucciones reproducibles y pendientes externos claramente identificados.

Control de versiones. El repositorio Git registra la incorporación del código, las pruebas y la documentación en cambios separados. El historial permite inspeccionar las versiones y recuperar estados anteriores mediante las herramientas estándar de Git. Consulta del historial: git log --oneline.

Definición de terminado técnico: cuatro tipos implementados, cálculos con precisión decimal, validaciones activas, pruebas aprobadas, exportación funcional y documentación verificable. La entrega académica completa añade los enlaces reales de GitHub y YouTube y la revisión de modalidad CIPA.

Limitaciones: demostración por consola con datos ficticios; sin persistencia, interfaz gráfica, prorrateo, obligaciones tributarias ni integración bancaria. Los costos reflejan únicamente los conceptos definidos para el ejercicio.

Conclusiones. La solución representa las cuatro modalidades mediante un contrato común y conserva sus diferencias de cálculo en clases especializadas. La política configurable permite resolver las ambigüedades del ejercicio sin dispersar porcentajes en el código. Las 25 pruebas aprobadas verifican resultados, fronteras y validaciones. La separación del dominio respecto de la presentación facilita agregar otras interfaces sin modificar las reglas existentes.

7. Ejecución, exposición y entrega

Requisitos: Python 3.10 o superior. No se requieren paquetes externos para ejecutar la nómina y sus pruebas. Desde la carpeta del proyecto:

Paso	Comando
Demostración	python -m nomina
Exportar CSV	python -m nomina --csv evidencia/nomina_ejemplo.csv
Pruebas	python -m unittest discover -s tests -v
Historial	git log --oneline

Exposición: se incluye una presentación breve de cinco diapositivas, con notas para una explicación de aproximadamente tres minutos. Resume objetivo, reglas, diseño SOLID, ejemplo numérico y pruebas. El guion ampliado permite profundizar en una demostración de ejecución si el docente lo solicita.

Repositorio: https://github.com/tab-Jesus/IngDeSoftware. Contiene el código, las pruebas, las evidencias y los materiales de presentación. La presentación animada puede abrirse localmente en un navegador.

Entrega en SIMA: confirmar la modalidad CIPA; grabar y publicar la exposición en YouTube; entregar el informe y los enlaces del repositorio y del video según las indicaciones del curso.

El acceso al curso SIMA no pudo verificarse con la herramienta disponible. Se trabajó con las dos páginas completas del PDF local suministrado. No se ha enviado ni publicado contenido en la plataforma académica.

Fuentes y material de consulta

Fuente de requisitos: *ACTIVIDAD UNIDAD 3.pdf*, documento proporcionado por el estudiante, dos páginas, sin autor ni fecha indicados. Reglas de negocio y requisitos de entrega.

Documentación técnica de consulta: Python Software Foundation, *decimal*, *unittest*, *abc* y *dataclasses*. Disponible en https://docs.python.org/3/library/ . Los resultados presentados provienen de la ejecución local, no de estos materiales de consulta.